

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
Engenharia da Computação
Compiladores

Analisador Léxico e tabela de símbolos

Alunos: Gabriel Bernalle Barbosa Matozinhos

Professora: Kecia Marques

Belo Horizonte, maio de 2025

Forma de uso do processador

O código abaixo a primeira parte (antes do pipe) serve para compilar e gerar o código objeto, a segunda parte roda o código. Nota-se que é preciso trocar o "testex.txt" pelo caso de teste que se deseja analisar (Entre 01 - 06).

```
javac .\Main.java | java .\Main.java .\examples\testex.txt
```

Descrição da abordagem utilizada na implementação, indicando as principais classes da aplicação e seus respectivos propósitos

O trabalho utiliza as classes Main, Lexeme, LexicalAnalysis, LexicalException, SymbolTable e TokenType . Adiante uma descrição simplificada sobre as principais.

Main

A função Main implementada tem como principal objetivo executar, testar e validar o funcionamento do analisador léxico desenvolvido. Sua abordagem é simples, direta e orientada ao tratamento dos arquivos de entrada, execução da análise léxica e exibição dos resultados.

Funcionamento Geral

O programa recebe como entrada um argumento de linha de comando, que corresponde ao caminho do arquivo fonte a ser analisado. Através da classe LexicalAnalysis, ele realiza a leitura sequencial do arquivo, tokenizando cada elemento do código fonte.

O funcionamento é estruturado de forma que, enquanto forem encontrados tokens válidos, eles são impressos no formato:

```
("token", TOKEN_TYPE)
```

Quando ocorre um erro léxico, um final de arquivo inesperado ou quando o fim do arquivo é alcançado corretamente, o programa realiza o tratamento adequado, exibindo mensagens claras e informativas.

Componentes da abordagem e seus propósitos

- Validação de entrada:
O programa verifica se foi fornecido exatamente um argumento (o nome do arquivo). Caso contrário, exibe uma mensagem de uso correto e encerra.

- Bloco Try-With-Resources:
O analisador léxico é instanciado dentro de um bloco try que garante o fechamento automático do recurso (LexicalAnalysis) após o uso, evitando vazamento de recursos como arquivos abertos.
- Loop de leitura de tokens:
O método nextToken() é chamado de forma iterativa, lendo token por token até que seja encontrado um token de parada, como END_OF_FILE, INVALID_TOKEN ou UNEXPECTED_EOF. Essa verificação é feita pela função auxiliar checkType().
- Impressão dos tokens:
Tokens válidos são impressos com seu lexema e tipo, facilitando a depuração e validação do funcionamento do analisador léxico.
- Tratamento de erros léxicos:
Se o analisador encontrar um token inválido (INVALID_TOKEN) ou um fim de arquivo inesperado (UNEXPECTED_EOF), o programa exibe uma mensagem de erro indicando a linha onde ocorreu o problema, ajudando na análise e correção do código fonte.
- Tratamento de exceções internas:
Possíveis falhas internas, como erros na abertura ou leitura de arquivos, são capturadas e tratadas, evitando que o programa termine de forma abrupta.

LexicalAnalysis

A implementação do analisador léxico foi desenvolvida utilizando a abordagem de máquina de estados finitos (autômato determinístico), combinada com uma tabela de símbolos. Esta abordagem permite a identificação e categorização de tokens em um código fonte de maneira eficiente, robusta e escalável.

Funcionamento Geral

O analisador lê o arquivo de entrada caractere por caractere, percorrendo diferentes estados conforme os padrões identificados. Cada estado representa uma situação específica no processo de reconhecimento dos tokens, como: operadores, delimitadores, identificadores, literais, números ou comentários.

Componentes principais da abordagem

- **Máquina de Estados Finitos:**
A base do analisador léxico. O fluxo de execução é controlado por um conjunto de estados e transições. Dependendo do caractere lido, a máquina muda de estado até reconhecer completamente um token ou identificar um erro léxico.
- **Tabela de Símbolos:**
Implementada como uma estrutura HashMap, essa tabela mapeia palavras-chave, operadores e símbolos para seus respectivos tipos de token (TokenType). A tabela permite rápida verificação se um lexema pertence a um token específico ou deve ser classificado como identificador (ID).

Propósitos dos principais estados

- **Estado 1 – Estado inicial:**
Realiza a leitura dos caracteres e direciona o fluxo para os estados responsáveis pelos diferentes tipos de token. Também ignora espaços, tabulações e quebras de linha (gerenciando o controle da contagem de linhas).
- **Estados de operadores e símbolos (4, 5, 6, 7, 8, 16):**
Reconhecem operadores lógicos (&&, ||), relacionais (>, >=, <, <=, !=, ==), aritméticos (+, -, *, /), além de símbolos como ;, :, (,), {, }.
- **Estados de comentários (2 e 3):**
Tratam comentários de linha (iniciados com %) e de bloco (delimitados por {}), descartando esses trechos do código durante a análise.
- **Estado 9 – Identificadores e palavras-chave:**
Reconhece sequências que podem ser tanto identificadores quanto palavras reservadas, fazendo a verificação na tabela de símbolos para diferenciar.
- **Estados 10 – Literais (strings):**
Reconhece textos delimitados por aspas ("), classificando-os corretamente como literais (LITERALS). Este estado também lida com erros, como literais não fechados ou inválidos.
- **Estados 11, 12 e 13 – Números:**
Diferenciam números inteiros (INTEGER_CONST) e números de ponto flutuante (FLOAT_CONST), realizando a transição correta após o ponto decimal.

- Estados finais (17 e 18):
O estado 18 representa um token reconhecido corretamente, enquanto o estado 17 trata situações de erro léxico, como tokens inválidos ou fim inesperado de arquivo.

SymbolTable

A classe SymbolTable (Tabela de Símbolos) desempenha um papel fundamental no analisador léxico, sendo responsável por armazenar e gerenciar os símbolos, operadores e palavras-chave da linguagem. Sua principal função é permitir que o analisador léxico reconheça rapidamente se um determinado lexema faz parte dos elementos reservados da linguagem.

Abordagem da implementação

- A SymbolTable foi implementada utilizando a estrutura de dados HashMap, do pacote java.util, que associa cada string (lexema) ao seu respectivo tipo (TokenType).
- A escolha do HashMap garante operações de busca eficientes, com complexidade média de $O(1)$, o que permite que a análise léxica aconteça de forma rápida e escalável, mesmo para códigos maiores.

Estrutura da Tabela

- Símbolos e delimitadores:
São inseridos na tabela lexemas como ;, :, ,, (,) e {}, que representam delimitadores e estruturas da linguagem.
- Operadores:
Incluem tanto operadores aritméticos (+, -, *, /, %) quanto operadores relacionais (==, !=, <, <=, >, >=) e operadores lógicos (&&, ||, !).
- Palavras-chave:
A tabela armazena todas as palavras reservadas da linguagem, como program, begin, if, then, end, while, do, repeat, until, entre outras. Isso permite distinguir identificadores comuns de palavras-chave.
- Outros símbolos:
Também são adicionados elementos especiais como aspas ("), aspas simples ('), e o underline (_), que possuem funções específicas dentro da

linguagem.

Métodos e seus propósitos

- `contains(String token)`
Verifica se determinado lexema existe na tabela de símbolos. Serve como uma verificação auxiliar para classificar se um token é um símbolo reservado ou não.
- `find(String token)`
Busca o tipo do token na tabela. Caso o lexema esteja presente, retorna seu respectivo `TokenType`. Se não estiver, considera-se que é um identificador (`TokenType.ID`), o que reflete um comportamento padrão de linguagens, onde qualquer sequência não reservada é tratada como identificador.

Apresentação dos Testes

Teste 01:

```
PS C:\Users\gbernal\Documents\Applications\projeto_compiladores> javac .\Main.java | java .\Main.java .\examples\teste01.txt
("program", PROGRAM)
("int", INT)
(":", TWOPOINTS)
("a", ID)
(",", COMMA)
("b", ID)
(",", COMMA)
("c", ID)
(";", SEMICOLON)
("float", FLOAT)
(":", TWOPOINTS)
("result", ID)
(";", SEMICOLON)
("char", CHAR)
(":", TWOPOINTS)
("ch", ID)
(";", SEMICOLON)
("begin", BEGIN)
("out", OUT)
("(", OP_ROUNDBRACK)
(" Digite o valor de a:", LITERALS)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("in", IN)
("(", OP_ROUNDBRACK)
("a", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("out", OUT)
("(", OP_ROUNDBRACK)
(" Digite o valor de c:", LITERALS)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("read", ID)
("(", OP_ROUNDBRACK)
```

```
("(", OP_ROUNDBRACK)
("ch", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("b", ID)
("=", ASSIGN)
("10", ID)
(";", SEMICOLON)
("result", ID)
("=", ASSIGN)
("(", OP_ROUNDBRACK)
("a", ID)
("*", MUL)
("ch", ID)
(")", CL_ROUNDBRACK)
("/", DIV)
("(", OP_ROUNDBRACK)
("b", ID)
("5", ID)
("-", SUB)
("345.27", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("out", OUT)
("(", OP_ROUNDBRACK)
("O resultado e: ", LITERALS)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("out", OUT)
("(", OP_ROUNDBRACK)
("result", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("result", ID)
("=", ASSIGN)
("result", ID)
("+", ADD)
("ch", ID)
(";", SEMICOLON)
("end", END)
("", END_OF_FILE)
```

Teste 02

```
PS C:\Users\gbernal\Documents\Applications\projeto_compiladores> javac .\Main.java | java .\Main.java .\examples\teste02.txt
("program", PROGRAM)
("float", FLOAT)
(":", TWOPOINTS)
("raio", ID)
(",", COMMA)
("area$", ID)
("=", ASSIGN)
("0.0", ID)
(";", SEMICOLON)
("begin", BEGIN)
("repeat", REPEAT)
("in", IN)
("(", OP_ROUNDBRACK)
("raio", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("char", CHAR)
(":", TWOPOINTS)
("resposta", ID)
(";", SEMICOLON)
("if", IF)
("(", OP_ROUNDBRACK)
("raio", ID)
(">", GREATER)
("0.0", ID)
("0.0", ID)
(")", CL_ROUNDBRACK)
("then", THEN)
("0.0", ID)
("0.0", ID)
(")", CL_ROUNDBRACK)
("0.0", ID)
(")", CL_ROUNDBRACK)
("0.0", ID)
(")", CL_ROUNDBRACK)
("then", THEN)
("0.0", ID)
("0.0", ID)
("0.0", ID)
```

```
(")", CL_ROUNDBRACK)
("then", THEN)
("area", ID)
("=", ASSIGN)
09: Lexema inválido [3. ]
```


Teste03

```
PS C:\Users\gbernal\Documents\Applications\projeto_compiladores> javac .\Main.java | java .\Main.java .\examples\teste03.txt
("program", PROGRAM)
("int", INT)
(":", TWOPOINTS)
("a", ID)
(", ", COMMA)
("b", ID)
(", ", COMMA)
("aux", ID)
(";", SEMICOLON)
("begin", BEGIN)
("in", IN)
("(", OP_ROUNDBRACK)
("a", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("in", IN)
("(", OP_ROUNDBRACK)
("b", ID)
(")", CL_ROUNDBRACK)
(";", SEMICOLON)
("if", IF)
("(", OP_ROUNDBRACK)
("a", ID)
(">", GREATER)
("b", ID)
(")", CL_ROUNDBRACK)
("then", THEN)
("int", INT)
("aux", ID)
(";", SEMICOLON)
("aux", ID)
("=", ASSIGN)
("b", ID)
(";", SEMICOLON)
("b", ID)
("=", ASSIGN)
("a", ID)
(";", SEMICOLON)
("a", ID)
("=", ASSIGN)
```

```
("aux", ID)
("end", END)
(";", SEMICOLON)
("out", OUT)
("(", OP_ROUNDBRACK)
("a", ID)
(";", SEMICOLON)
("out", OUT)
("(", OP_ROUNDBRACK)
("b", ID)
(")", CL_ROUNDBRACK)
("end", END)
("", END_OF_FILE)
```

Teste04

```
PS C:\Users\gbernal\Documents\Applications\projeto_compiladores> javac .\Main.java | java .\Main.java .\examples\teste04.txt
("program", PROGRAM)
("a", ID)
(", ", COMMA)
("b", ID)
(", ", COMMA)
("c", ID)
(", ", COMMA)
("maior", ID)
(", ", COMMA)
("outro", ID)
(":", TWOPPOINTS)
("int", INT)
("; ", SEMICOLON)
("begin", BEGIN)
("repeat", REPEAT)
("out", OUT)
("(" , OP_ROUND BRACK)
("A", LITERALS)
(")", CL_ROUND BRACK)
("; ", SEMICOLON)
("in", IN)
("(" , OP_ROUND BRACK)
("a", ID)
(")", CL_ROUND BRACK)
("; ", SEMICOLON)
("out", OUT)
("(" , OP_ROUND BRACK)
("B", LITERALS)
(")", CL_ROUND BRACK)
("; ", SEMICOLON)
("in", IN)
("(" , OP_ROUND BRACK)
("b", ID)
(")", CL_ROUND BRACK)
("; ", SEMICOLON)
("out", OUT)
("(" , OP_ROUND BRACK)
("C", LITERALS)
(")", CL_ROUND BRACK)
("; ", SEMICOLON)
```

```
(";", SEMICOLON)
("in", IN)
("(" , OP_ROUND BRACK)
("c", ID)
(")", CL_ROUND BRACK)
("; ", SEMICOLON)
("%if", ID)
("(" , OP_ROUND BRACK)
("(" , OP_ROUND BRACK)
("a", ID)
(">", GREATER)
("b", ID)
(")", CL_ROUND BRACK)
("and", ID)
("(" , OP_ROUND BRACK)
("a", ID)
(">", GREATER)
("c", ID)
(")", CL_ROUND BRACK)
(")", CL_ROUND BRACK)
("end", END)
("maior", ID)
("=", ASSIGN)
("a", ID)
("else", ELSE)
("if", IF)
("(" , OP_ROUND BRACK)
("b", ID)
(">", GREATER)
("c", ID)
(")", CL_ROUND BRACK)
("then", THEN)
("maior", ID)
("=", ASSIGN)
("b", ID)
("; ", SEMICOLON)
("else", ELSE)
("maior", ID)
("=", ASSIGN)
```

```
("c", ID)
("end", END)
("end", END)
(";", SEMICOLON)
("out", OUT)
("(" , OP_ROUND BRACK)
("""Maior valor: """, LITERALS)
23: Fim de arquivo inesperado
```

Teste 05

```
PS C:\Users\gbernal\Documents\Applications\projeto_compiladores> javac .\Main.java | java .\Main.java .\examples\teste05.txt
("programa", ID)
("declare", ID)
("inteiro", ID)
(":", TWOPOINTS)
("pontuacao", ID)
(",", COMMA)
("pontuacaoMaxima", ID)
(",", COMMA)
("disponibilidade", ID)
(";", SEMICOLON)
("char", CHAR)
(":", TWOPOINTS)
("pontuacaoMinima", ID)
("begin", BEGIN)
("disponibilidade", ID)
("=", ASSIGN)
("'s'", CHAR_CONST)
(";", SEMICOLON)
("pontuacaoMinima", ID)
("=", ASSIGN)
("50", ID)
(";", SEMICOLON)
("pontuacaoMaxima", ID)
("=", ASSIGN)
("100", ID)
(";", SEMICOLON)
("out", OUT)
("(" , OP_ROUND BRACK)
("""Pontuacao Candidato: """, LITERALS)
(")", CL_ROUND BRACK)
(";", SEMICOLON)
("in", IN)
("(" , OP_ROUND BRACK)
```

```
("""Disponibilidade Candidato: """, LITERALS)
(")", CL_ROUND BRACK)
(";", SEMICOLON)
("in", IN)
("(" , OP_ROUND BRACK)
("disponibilidade", ID)
(")", CL_ROUND BRACK)
(";", SEMICOLON)
("{", END_OF_FILE)
```

Teste 06

```
PS C:\Users\gbernal\Documents\Applications\projeto_compiladores> javac .\Main.java | java .\Main.java .\examples\teste06.txt
("program", PROGRAM)
("exemplo", ID)
("int", INT)
(":", TWOPOINTS)
("x", ID)
(",", COMMA)
("y", ID)
(",", COMMA)
("z", ID)
("float", FLOAT)
(":", TWOPOINTS)
("media", ID)
("char", CHAR)
(":", TWOPOINTS)
("letra", ID)
("begin", BEGIN)
("x", ID)
("=", ASSIGN)
("10", ID)
("y", ID)
("=", ASSIGN)
("20", ID)
("z", ID)
("=", ASSIGN)
("x", ID)
("+", ADD)
("y", ID)
("media", ID)
("=", ASSIGN)
("(", OP_ROUNDBRACK)
("x", ID)
("+", ADD)
("y", ID)
("+", ADD)
("z", ID)
(")", CL_ROUNDBRACK)
("/", DIV)
("3.0", ID)
("if", IF)
```

```
("media", ID)
(">=", GREATER_EQUAL)
("15", ID)
("then", THEN)
("out", OUT)
("(", OP_ROUNDBRACK)
("Aprovado", LITERALS)
(")", CL_ROUNDBRACK)
("else", ELSE)
("out", OUT)
("(", OP_ROUNDBRACK)
("Reprovado", LITERALS)
(")", CL_ROUNDBRACK)
("end", END)
("repeat", REPEAT)
("in", IN)
("(", OP_ROUNDBRACK)
("x", ID)
(")", CL_ROUNDBRACK)
("x", ID)
("=", ASSIGN)
("x", ID)
("-", SUB)
("1", ID)
("until", UNTIL)
("x", ID)
("<=", LOWER_EQUAL)
("0", ID)
("end", END)
("", END_OF_FILE)
```